

```
+++
date = '2026-04-14T06:11:20+08:00'
title = 'STL'
+++
```

# STL 容器总览

## 1. STL 容器是什么（核心一句话）

**STL 容器 = 模板类 + 数据结构封装**  
提供统一接口来存储、管理数据，并与迭代器、算法共同构成 STL 三大组件。

## 2. 容器分类与特点

### 2.1 顺序容器（Sequence Containers）

序列容器（Sequence Containers）是 STL 中的一类容器，特点是 元素按照严格的线性顺序存储，支持顺序访问和插入删除操作。主要包括：vector、deque、list、forward\_list、array。

按插入顺序存储，强调顺序与遍历效率。

| 容器           | 底层结构   | 随机访问   | 插入/删除     | 典型场景   |
|--------------|--------|--------|-----------|--------|
| vector       | 动态数组   | ✓ O(1) | 尾部快，其他位置慢 | 大量随机访问 |
| deque        | 分段连续空间 | ✓ O(1) | 头尾都快      | 双端操作频繁 |
| list         | 双向链表   | ✗      | 任意位置 O(1) | 频繁插入删除 |
| forward_list | 单向链表   | ✗      | O(1)      | 更轻量的链表 |

面试要点：

- vector 扩容：**指数扩容（通常 ×2）**，导致迭代器失效
- list/forward\_list：**不支持随机访问**，迭代器稳定
- deque：**不是连续内存**，但支持随机访问

### 2.2 关联容器（Associative Containers）

基于 **红黑树（RB-tree）**，自动排序，log 级别查找。

| 容器       | 是否允许重复 | 存储内容      | 是否排序 |
|----------|--------|-----------|------|
| set      | 否      | key       | ✓    |
| multiset | 是      | key       | ✓    |
| map      | 否      | key-value | ✓    |
| multimap | 是      | key-value | ✓    |

面试要点：

- 查找、插入、删除： $O(\log n)$
- key 是 const，不可修改
- 迭代器稳定性强（删除当前元素除外）

## 2.3 无序关联容器（Unordered Containers）

基于 哈希表，强调平均  $O(1)$  查找。

| 容器                 | 是否允许重复 | 底层结构 | 查找复杂度     |
|--------------------|--------|------|-----------|
| unordered_set      | 否      | 哈希表  | 平均 $O(1)$ |
| unordered_multiset | 是      | 哈希表  | 平均 $O(1)$ |
| unordered_map      | 否      | 哈希表  | 平均 $O(1)$ |
| unordered_multimap | 是      | 哈希表  | 平均 $O(1)$ |

面试要点：

- 最坏情况退化为  $O(n)$ （哈希冲突严重）
- 迭代器遍历顺序不稳定
- 自定义 key 需要提供 hash + equal\_to

## 2.4 容器适配器（Container Adapters）

不是新容器，只是对底层容器的封装。

| 适配器   | 默认底层容器 | 特点   |
|-------|--------|------|
| stack | deque  | LIFO |
| queue | deque  | FIFO |

| 适配器            | 默认底层容器        | 特点        |
|----------------|---------------|-----------|
| priority_queue | vector + heap | 取最大/最小优先级 |

面试要点：

- priority\_queue 默认是 大顶堆
- 可以通过自定义比较器变成小顶堆

### 3. 总结（面试背诵版）

STL 容器分为顺序容器、关联容器、无序容器和容器适配器。  
顺序容器强调插入顺序；关联容器基于红黑树，强调有序键值查找；无序容器基于哈希表，强调平均  $O(1)$  查找；适配器是对现有容器的功能封装。  
容器与迭代器、算法共同构成 STL 的三大核心组件。

## 3.2 STL 容器的常见操作与复杂度（Complexity Overview）

STL 为所有容器提供了统一的接口，但不同容器的底层结构不同，因此操作复杂度也不同。掌握复杂度是面试高频点。

### 3.2.1 常见操作分类

STL 容器的典型操作包括：

- 插入 (insert / push\_back / emplace)
- 删除 (erase / pop\_back / pop\_front)
- 查找 (find / count / lower\_bound / operator[])
- 遍历 (iterator)
- 访问 (front / back / at / operator[])

### 3.2.2 各容器操作复杂度总表

| 容器     | 插入                       | 删除        | 查找     | 随机访问     | 备注        |
|--------|--------------------------|-----------|--------|----------|-----------|
| vector | 尾部 $O(1)$ ;<br>中间 $O(n)$ | 中间 $O(n)$ | $O(n)$ | ✓ $O(1)$ | 扩容导致迭代器失效 |
| deque  | 头尾 $O(1)$                | 头尾 $O(1)$ | $O(n)$ | ✓ $O(1)$ | 分段连续      |

| 容器             | 插入            | 删除           | 查找       | 随机访问 | 备注          |
|----------------|---------------|--------------|----------|------|-------------|
| list           | 任意位置 O(1)     | 任意位置 O(1)    | O(n)     | ✗    | 迭代器稳定       |
| forward_list   | O(1)          | O(1)         | O(n)     | ✗    | 单链表         |
| set / multiset | O(log n)      | O(log n)     | O(log n) | ✗    | 红黑树有序       |
| map / multimap | O(log n)      | O(log n)     | O(log n) | ✗    | key 有序      |
| unordered_set  | 平均 O(1)       | 平均 O(1)      | 平均 O(1)  | ✗    | 哈希冲突退化 O(n) |
| unordered_map  | 平均 O(1)       | 平均 O(1)      | 平均 O(1)  | ✗    | 哈希表         |
| stack          | O(1)          | O(1)         | ✗        | ✗    | 适配器         |
| queue          | O(1)          | O(1)         | ✗        | ✗    | 适配器         |
| priority_queue | push O(log n) | pop O(log n) | top O(1) | ✗    | 堆结构         |

### 3.2.3 迭代器失效规则（高频陷阱）

- **vector**：扩容、插入、删除 → 所有迭代器失效
- **deque**：头尾插入可能导致部分迭代器失效
- **list / forward\_list**：插入删除不影响其他迭代器
- **map / set**：插入不失效；删除当前迭代器失效
- **unordered\_map / unordered\_set**：rehash 时所有迭代器失效

## 3.3 STL 容器的选择策略（如何选容器）

面试常问：“这个场景你会用哪个容器？”

核心是根据 **访问模式 + 插入删除模式 + 是否需要排序 + 是否需要唯一性** 来选择。

### 3.3.1 根据访问方式选择

- 需要随机访问 → **vector / deque**

- 不需要随机访问 → list / forward\_list / map / unordered\_map

### 3.3.2 根据插入删除模式选择

- 频繁在中间插入删除 → list
- 频繁头尾插入删除 → deque
- 只在尾部插入 → vector

### 3.3.3 根据查找需求选择

- 需要有序查找 → map / set（红黑树）
- 需要最快查找 → unordered\_map / unordered\_set（哈希）
- 需要范围查询（lower\_bound） → map / set

### 3.3.4 根据是否需要排序

- 自动排序 → map / set
- 不需要排序 → unordered\_map / unordered\_set
- 需要自定义排序 → priority\_queue / sort + vector

### 3.3.5 根据是否允许重复

- 允许重复 → multiset / multimap / unordered\_multiset / unordered\_multimap
- 不允许重复 → set / map / unordered\_set / unordered\_map

## 3.4 STL 容器底层实现（数据结构视角）

理解底层结构是面试官判断你是否“真正懂 STL”的关键。

### 3.4.1 顺序容器底层结构

| 容器           | 底层结构                 | 特点          |
|--------------|----------------------|-------------|
| vector       | 连续动态数组               | 扩容 ×2，随机访问快 |
| deque        | 分段连续数组（map + buffer） | 头尾插入快       |
| list         | 双向链表                 | 稳定迭代器       |
| forward_list | 单向链表                 | 更轻量         |

### 3.4.2 关联容器底层结构

| 容器                  | 底层结构         | 特点          |
|---------------------|--------------|-------------|
| set / map           | 红黑树（RB-tree） | 自动排序，log 查找 |
| multiset / multimap | 红黑树          | 允许重复        |

红黑树性质：

- 自平衡
- 高度  $O(\log n)$
- 中序遍历有序

### 3.4.3 无序容器底层结构

| 容器                            | 底层结构                 | 特点           |
|-------------------------------|----------------------|--------------|
| unordered_map / unordered_set | 哈希表（bucket + 链表/开链法） | 平均 $O(1)$ 查找 |

关键点：

- 哈希冲突 → 链表或拉链法
- 负载因子过高 → rehash（迭代器全部失效）

### 3.4.4 容器适配器底层结构

| 适配器            | 默认底层容器        | 底层结构 |
|----------------|---------------|------|
| stack          | deque         | 双端队列 |
| queue          | deque         | 双端队列 |
| priority_queue | vector + heap | 二叉堆  |

## 优先级队列（priority\_queue）

### 定义

std::priority\_queue 是 C++ STL 提供的 **堆（heap）容器适配器**。

- 保证队首元素始终是 **优先级最高** 的元素
- 默认是 **最大堆**（最大值在队首）
- 可通过比较器自定义为最小堆或其他排序规则

一句话：

**priority\_queue = 封装好的堆，支持快速取最大/最小值。**

## 底层数据结构

- 底层容器：**vector**
- 维护方式：**堆算法**（`make_heap`、`push_heap`、`pop_heap`）
- 默认比较器：`std::less<T>` → **大顶堆**

核心结构：

`vector + heap = priority_queue`

# 定义方式

```
#include <queue>
#include <vector>
#include <functional>
using namespace std;

// 默认：最大堆
priority_queue<int> maxHeap;

// 最小堆
priority_queue<int, vector<int>, greater<int>> minHeap;

// 自定义比较器
struct cmp {
    bool operator()(int a, int b) const {
        return a > b; // 小顶堆
    }
};
priority_queue<int, vector<int>, cmp> customHeap;
```

# 常用操作与复杂度

| 操作      | 说明          | 时间复杂度    |
|---------|-------------|----------|
| push(x) | 插入元素        | O(log n) |
| pop()   | 删除队首（最大/最小） | O(log n) |
| top()   | 访问队首元素      | O(1)     |
| empty() | 判空          | O(1)     |
| size()  | 返回元素个数      | O(1)     |

一句话：

**priority\_queue 插入/删除是 log n，取最大/最小是 O(1)。**



# 使用场景

- **调度系统**：总是取优先级最高的任务
- **Dijkstra 最短路径**：用最小堆维护当前最短距离
- **A\* 搜索**：根据估价函数取最优节点
- **霍夫曼编码**：反复取最小频率节点
- **数据流中位数**：大顶堆 + 小顶堆
- **Top-K 问题**：用最小堆维护前 K 大元素

一句话：

凡是“每次取最大/最小”的场景，都适合 `priority_queue`。

# 总结

- `priority_queue` 本质是 **堆的封装**
- 默认 **大顶堆**，可通过比较器改成 **小顶堆**
- 插入/删除  $O(\log n)$ ，访问队首  $O(1)$
- 在图算法、调度、Top-K、数据流处理中非常常用

如果你愿意，我还能帮你整理：

- `priority_queue` 如何自定义结构体排序
- `priority_queue` 与 `multiset` 的区别
- 手写堆（heap）的实现原理
- Top-K 问题的最佳解法总结

告诉我你想继续哪一部分，我可以帮你写得更深入。